

# Introduction to Machine Learning

## Chapter03: Classification

---

PAISIT KHANARSA, PHD

INSTITUTE OF FIELD ROBOTICS (FIBO)

KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI (KMUTT)

# Outlines

---

- Motivation
- Logistic regression model
- Classification cost function
- Classification Performance

# Motivation

---

- Suppose there is a fish canning factory which processes two types of fish Sardine and Trout



- Currently, human labours are needed to process the fish.

# Motivation

---

It might be more cost effective if we have a robotic arm which can 'automatically' separate sardine and trout.



(a) sardine



(b) trout

# Motivation

---

How do we program such a robot ?

Classical approach: encode everything we know about the fish in our code

---

```
if 10 < length < 30 and 1 < weight < 5:  
    print("Sardine")  
else:  
    print("Trout")
```

---

Problems ?

# Motivation

---

## Problems with classical approach

- Our knowledge is incomplete (low accuracy)
- Our robot program might end up being super complicated or our robot is not adaptive
  - In the future sardine, in general, might get smaller

# Motivation

---

## Alternative approach

- Let's extract some physical characteristics from sardine and trout
  - e.g., length, weight, colour, existence of body patterns, etc
- And find relationship between physical characteristics and fish types
- Mathematically, find a function which maps from physical characteristics (features) to fish category (label)
- This is a machine learning based classification approach

# Motivation

---

## Definition

Given a set of features/label pairs  $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ , where  $\mathbf{x}_i \in \mathbb{R}^m$  and  $y_i \in \{1, \dots, K\}$ , a classification task is a task of **inferring a real-valued function**  $f(\mathbf{x})$  which maps features to its corresponding class label with **high accuracy**.

# Motivation

---

- $f()$  is called a classifier
- Each classification model has different functional form  $f()$  from the others
- Adjusting function parameters is accounted for the learning part
- There exist classifiers which do not assume function form such as k-NN, Decision Tree. Such classifiers are called non-parametric models

# Classification vs Regression

---

## Regression

- ▶ For  $(\mathbf{x}, y)$ , the output  $y$  is called the target and  $y \in \mathbb{R}$

## Classification

- ▶ For  $(\mathbf{x}, y)$ , the output  $y$  is called the class label and  $y \in \mathcal{I}$ 
  - ▶ Output from regression model can be transformed into label prediction by thresholding

Both belong to supervised learning paradigm

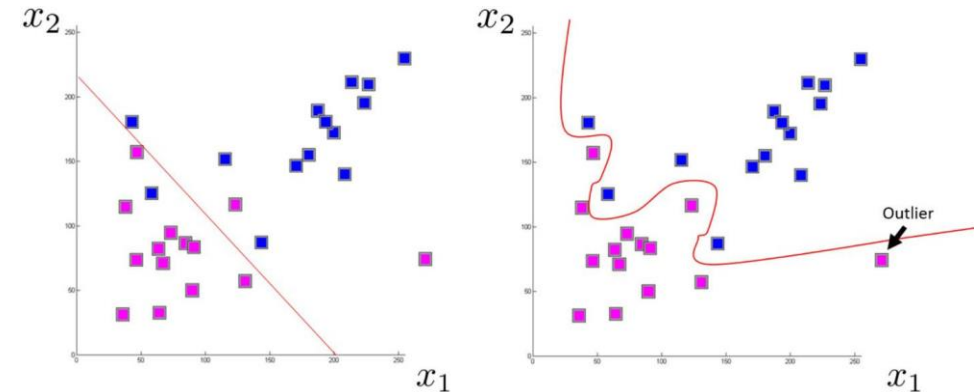
# Types of classifier by functional form

## Linear

- ▶ Assumes data is separable using linear decision boundary
- ▶ Least Square Classifier, Logistic regression, Support Vector Machine, Neural networks without hidden layer

## Non-linear

- ▶ Assumes data is separable using non-linear decision boundary
- ▶ SVM (with kernel), Trees, Neural networks, Ensemble of classifiers (boosting)



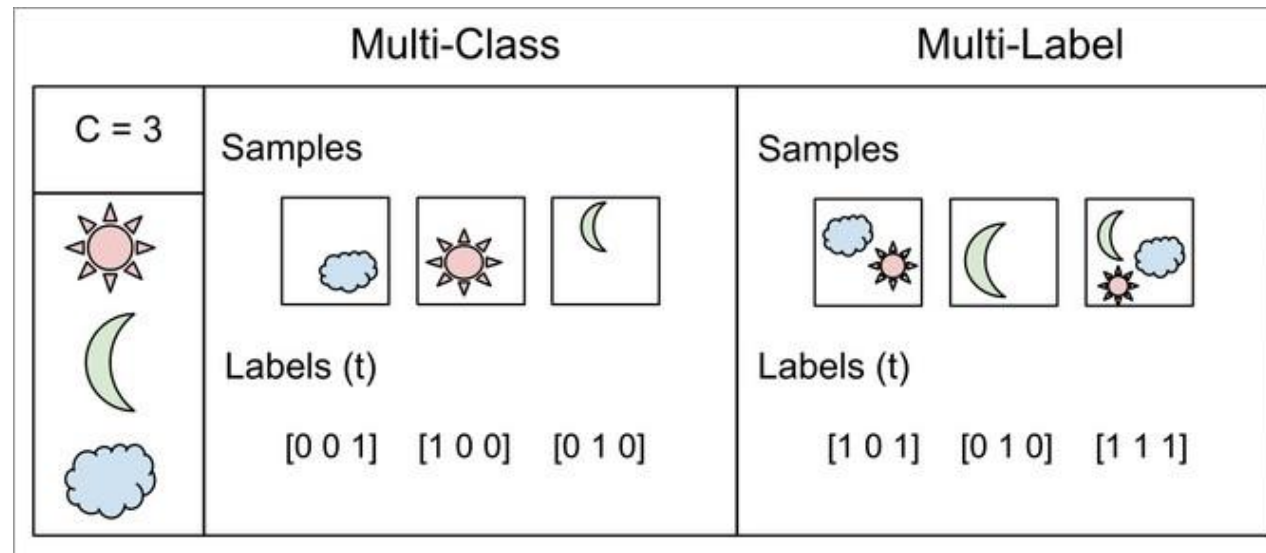
# Classification tasks/applications

---

- Binary classification
- Multiclass classification
- Multi-label classification
- One-class classification

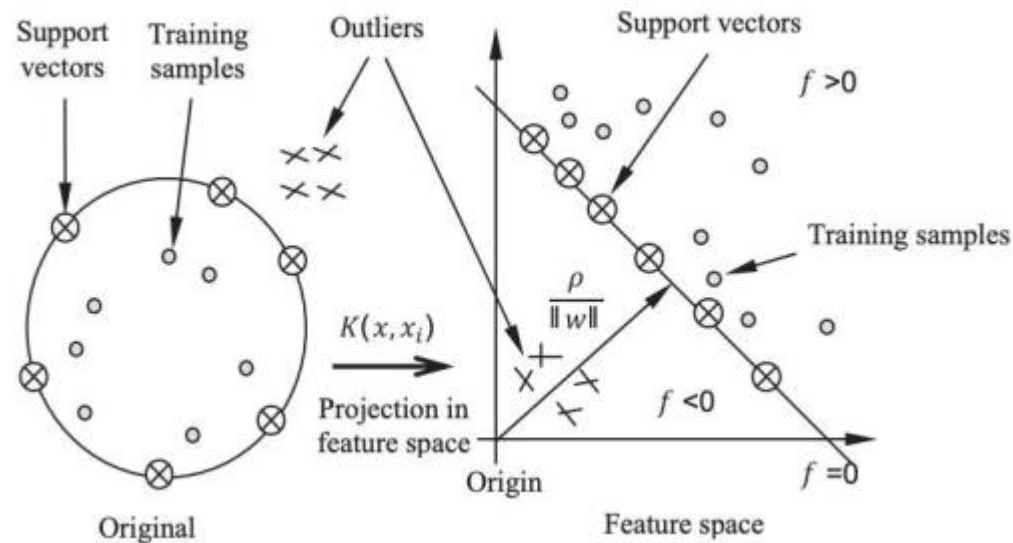
# Multi-class vs multi-label

- Multi-class: there are  $K > 2$  data classes and one input can belong to only one class
- Multi-label: there are  $K > 2$  data classes and one input can belong to more than one class
- Application: medical diagnosis etc.



# One class classification

- Application: person identification, object detection
- Negative examples are often countless: impractical to use binary classification



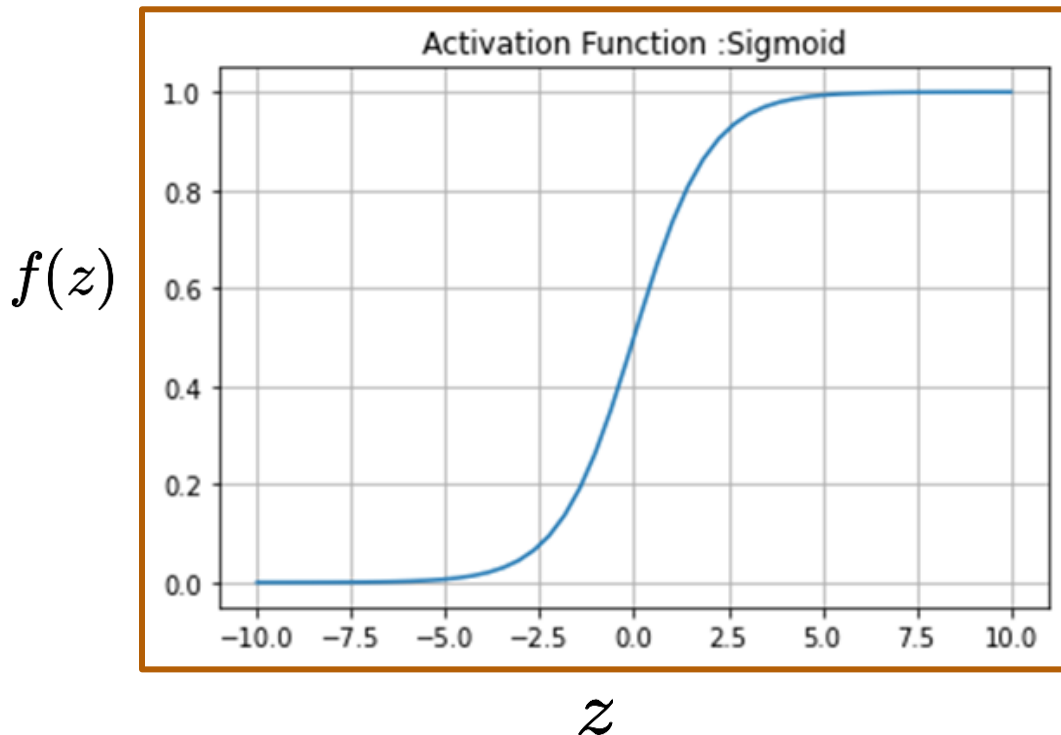
<https://www.sciencedirect.com/science/article/abs/pii/S0031320314002751>

# Logistic regression model

---

# Logistic regression model

Logistic regression is similar to linear regression, however it is used for classification problem.



$$f(z) = h(\vec{x}) = f(\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n)$$

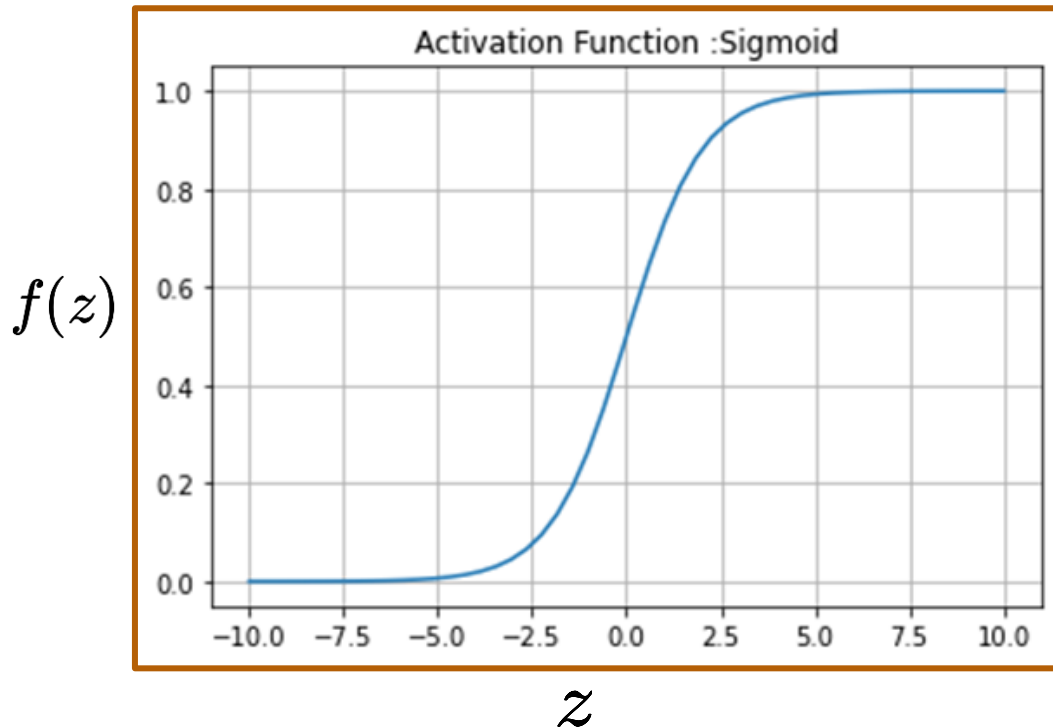
$$f(z) = \frac{1}{1+e^{-z}}$$

- ❖ Domain:  $z \in \mathfrak{R}$
- ❖ Range:  $f(z) \in (0, 1)$
- ❖  $z \rightarrow -\infty$  then  $f(z) \rightarrow 0$
- ❖  $z \rightarrow \infty$  then  $f(z) \rightarrow 1$

# Logistic regression model

Logistic regression is similar to linear regression, however it is used for classification problem.

$$f(z) = h(\vec{x}) = f(\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n)$$



Probability

|            |                                          |     |                    |
|------------|------------------------------------------|-----|--------------------|
| Sample 1   | $x_1^{(1)}, x_2^{(1)}, \dots, x_n^{(1)}$ | ... | $h(\vec{x}^{(1)})$ |
| Sample 2   | $x_1^{(2)}, x_2^{(2)}, \dots, x_n^{(2)}$ | ... | $h(\vec{x}^{(2)})$ |
| ...        | ...                                      | ... | ...                |
| Sample $i$ | $x_1^{(i)}, x_2^{(i)}, \dots, x_n^{(i)}$ | ... | $h(\vec{x}^{(i)})$ |

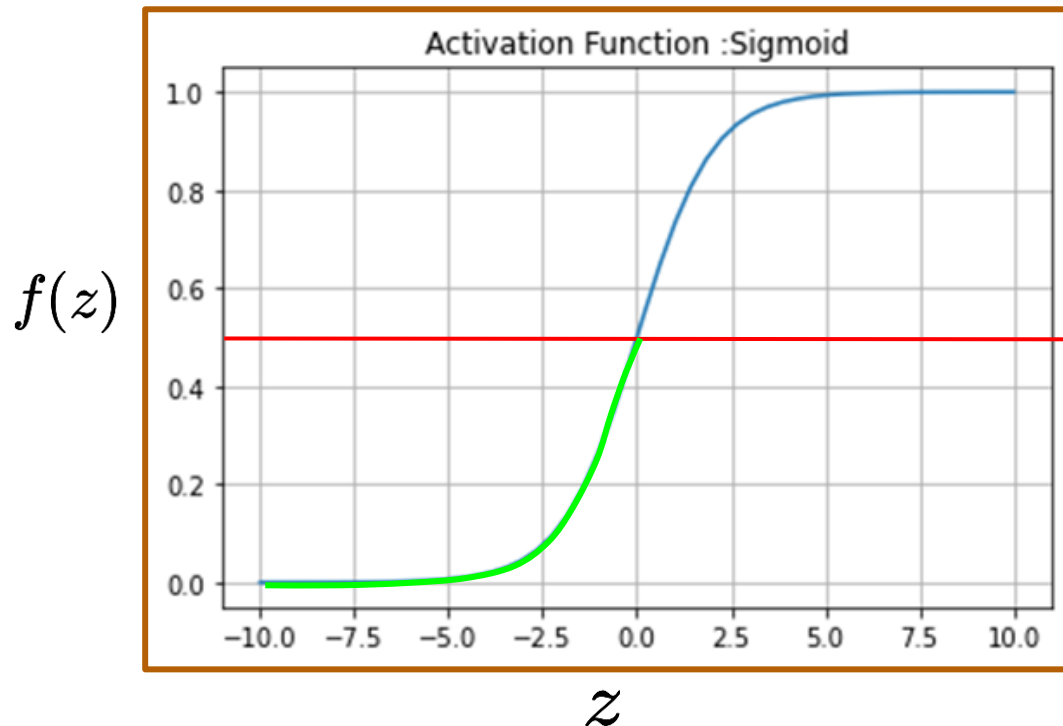
if  $h(\vec{x}^{(i)}) = 0.8$  :

Sample  $i \equiv$  Class 1 : 80% or 0.8

Sample  $i \equiv$  Class 0 : 20% or 0.2

# Logistic regression model

Logistic regression is similar to linear regression, however it is used for classification problem.



$$f(z) = h(\vec{x}) = f(\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n)$$

## Example

| Sample $i$ | $x_1^i$ | $x_2^i$ | ... | $h(\vec{x}^{(i)})$ | $y$ |
|------------|---------|---------|-----|--------------------|-----|
| 1          | 3       | A       | ... | 0.6                | 1   |
| 2          | 6       | A       | ... | 0.3                | 0   |
| 3          | 12      | B       | ... | 0.5                | 1   |

*Decision rule :*

$$y = \begin{cases} 1, & \text{if } f(z) = h(\vec{x}^{(i)}) \geq 0.5 \text{ or } z \geq 0 \\ 0, & \text{if } f(z) = h(\vec{x}^{(i)}) < 0.5 \text{ or } z < 0 \end{cases}$$

# A simple example

---

Let's look at a simple logistic regression procedure.

| Sample<br>$i$ | age<br>$x_1^{(i)}$ | income<br>$x_2^{(i)}$ | dependents<br>$x_3^{(i)}$ | spending<br>$x_4^{(i)}$ | history<br>$y$ |
|---------------|--------------------|-----------------------|---------------------------|-------------------------|----------------|
| 1             | 40                 | 50                    | 0                         | 30                      | 1              |
| 2             | 25                 | 40                    | 2                         | 35                      | 1              |
| 3             | 18                 | 10                    | 0                         | 12                      | 0              |
| 4             | 34                 | 22                    | 1                         | 10                      | 1              |
|               | 55                 | 65                    | 4                         | 60                      | ?              |

# A simple example

---

We first to do preprocessing, such as normalization and standardization

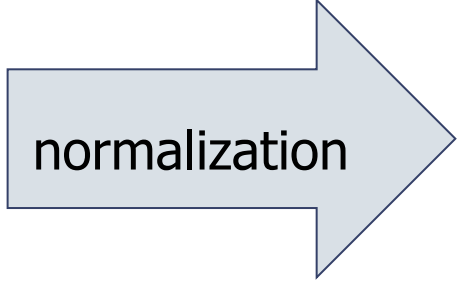
| $i$ | $x_1^{(i)}$ | $x_2^{(i)}$ | $x_3^{(i)}$ | $x_4^{(i)}$ | $y$ |
|-----|-------------|-------------|-------------|-------------|-----|
| 1   | 40          | 50          | 0           | 30          | 1   |
| 2   | 25          | 40          | 2           | 35          | 1   |
| 3   | 18          | 10          | 0           | 12          | 0   |
| 4   | 34          | 22          | 1           | 10          | 1   |

# A simple example

We first to do preprocessing, such as normalization and standardization

$$new\ x_j^{(i)} = \frac{x_j^{(i)}}{\max(x_j^{(i)} \mid i=1,2,3,4\ and\ j=1,2,3,4)}$$

| $i$ | $x_1^{(i)}$ | $x_2^{(i)}$ | $x_3^{(i)}$ | $x_4^{(i)}$ | $y$ |
|-----|-------------|-------------|-------------|-------------|-----|
| 1   | 40          | 50          | 0           | 30          | 1   |
| 2   | 25          | 40          | 2           | 35          | 1   |
| 3   | 18          | 10          | 0           | 12          | 0   |
| 4   | 34          | 22          | 1           | 10          | 1   |



normalization

| $i$ | $x_1^{(i)}$ | $x_2^{(i)}$ | $x_3^{(i)}$ | $x_4^{(i)}$ | $y$ |
|-----|-------------|-------------|-------------|-------------|-----|
| 1   | 0.44        | 0.63        | 0           | 0.6         | 1   |
| 2   | 0.28        | 0.5         | 0.5         | 0.7         | 1   |
| 3   | 0.2         | 0.13        | 0           | 0.24        | 0   |
| 4   | 0.38        | 0.28        | 0.25        | 0.2         | 1   |

$$\max(x_j^{(i)} \mid i = 1, 2, 3, 4\ and\ j = 1, 2, 3, 4) = 91$$

# A simple example

---

Then, fit the logistic regression to the data. Suppose after fitting, here are the weight numbers.

$$\begin{aligned}h(\vec{x}) &= f(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4) \\&= f(0 + 0.7x_1 + 0.6x_2 - 0.1x_3 - 0.2x_4)\end{aligned}$$

# A simple example

---

$$\begin{aligned}h(\vec{x}) &= f(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4) \\ &= f(0 + 0.7x_1 + 0.6x_2 - 0.1x_3 - 0.2x_4)\end{aligned}$$

Let make prediction for a single customer.

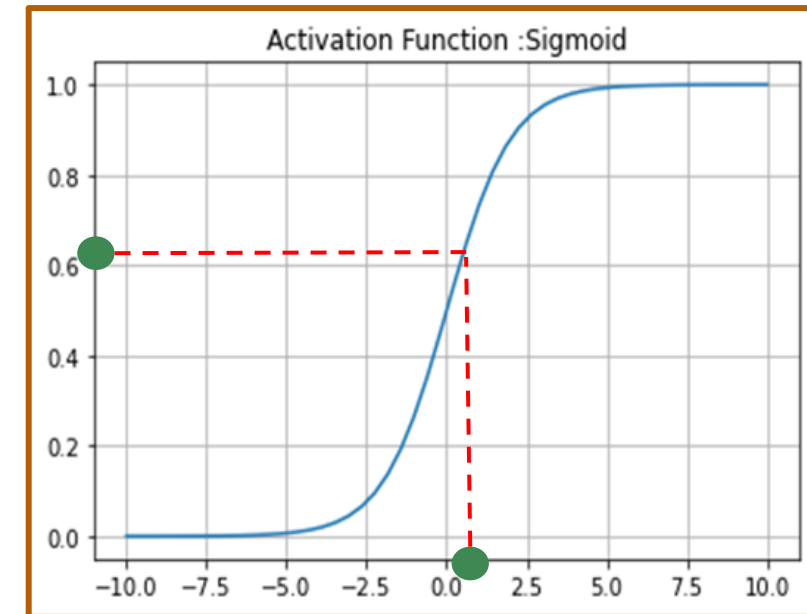
| $j$             | Raw data | Normalization | $\theta_j$   | $\theta_j x_j$ |
|-----------------|----------|---------------|--------------|----------------|
| age $x_1$       | 40.00    | 0.44          | 0.7          | 0.31           |
| income $x_2$    | 57.00    | 0.63          | 0.6          | 0.38           |
| dependent $x_3$ | 0.00     | 0.00          | -0.1         | 0.00           |
| spending $x_4$  | 55       | 0.60          | -0.2         | -0.12          |
|                 |          |               | Sum:         | 0.57           |
|                 |          |               | $h(\vec{x})$ | 0.64           |

# A simple example

$$\begin{aligned}h(\vec{x}) &= f(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4) \\ &= f(0 + 0.7x_1 + 0.6x_2 - 0.1x_3 - 0.2x_4)\end{aligned}$$

Let make prediction for a single customer.

| $j$             | Raw data | Normalization | $\theta_j$   | $\theta_j x_j$ |
|-----------------|----------|---------------|--------------|----------------|
| age $x_1$       | 40.00    | 0.44          | 0.7          | 0.31           |
| income $x_2$    | 57.00    | 0.63          | 0.6          | 0.38           |
| dependent $x_3$ | 0.00     | 0.00          | -0.1         | 0.00           |
| spending $x_4$  | 55       | 0.60          | -0.2         | -0.12          |
|                 |          |               | Sum:         | 0.57           |
|                 |          |               | $h(\vec{x})$ | 0.64           |



# A simple example

$$\begin{aligned}h(\vec{x}) &= f(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4) \\ &= f(0 + 0.7x_1 + 0.6x_2 - 0.1x_3 - 0.2x_4)\end{aligned}$$

Let make prediction for a single customer.

| $j$             | Raw data | Normalization | $\theta_j$   | $\theta_j x_j$ |
|-----------------|----------|---------------|--------------|----------------|
| age $x_1$       | 40.00    | 0.44          | 0.7          | 0.31           |
| income $x_2$    | 57.00    | 0.63          | 0.6          | 0.38           |
| dependent $x_3$ | 0.00     | 0.00          | -0.1         | 0.00           |
| spending $x_4$  | 55       | 0.60          | -0.2         | -0.12          |
|                 |          |               | Sum:         | 0.57           |
|                 |          |               | $h(\vec{x})$ | 0.64           |

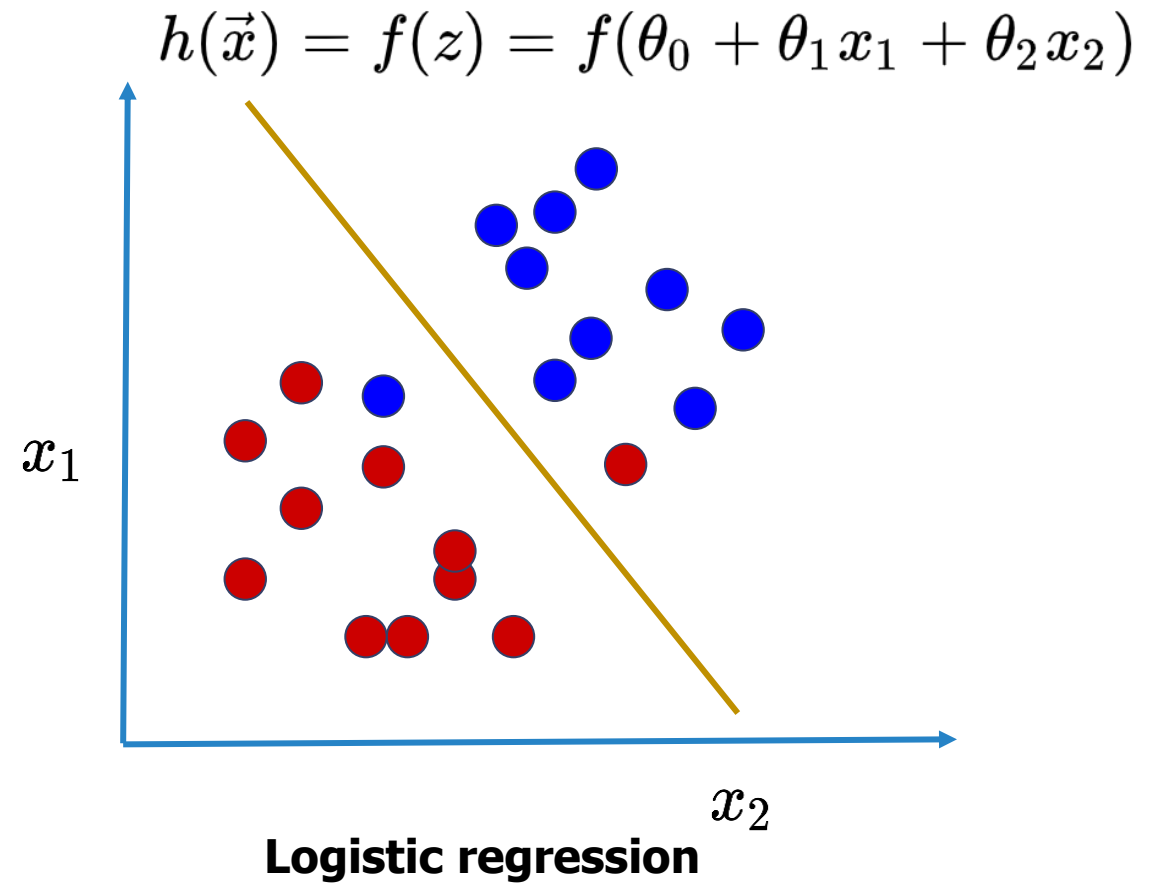
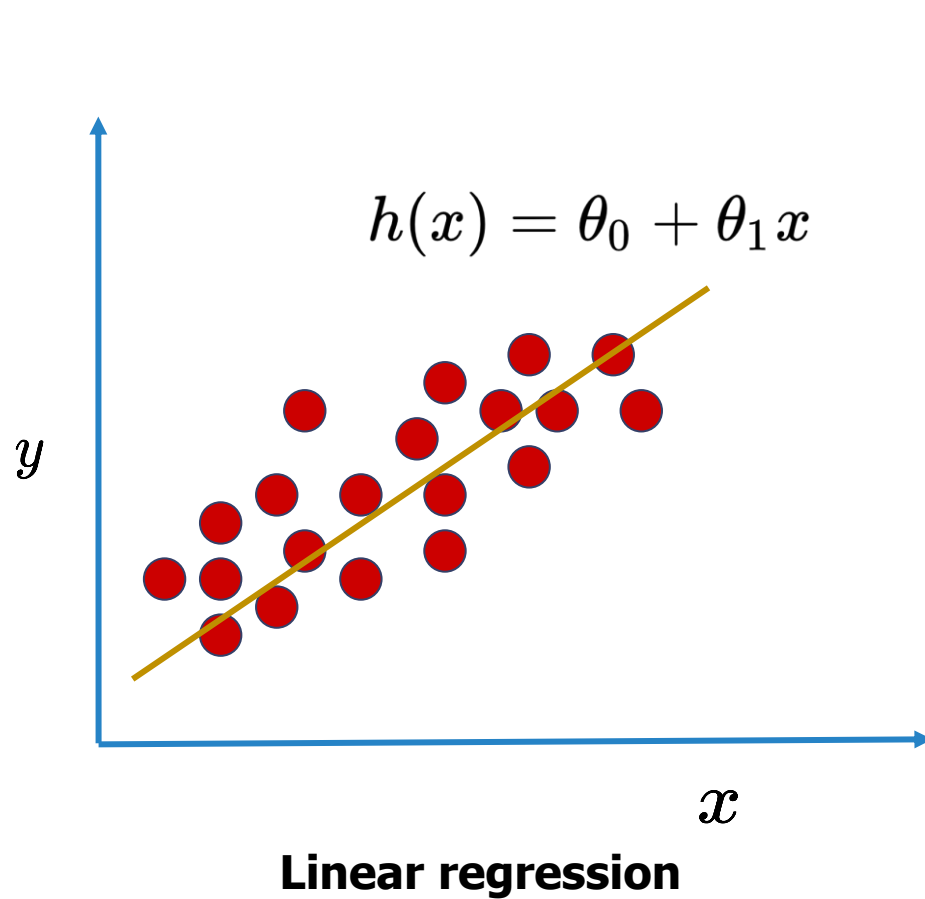
$h(\vec{x})$  Indicates the probability of customer being good.

*Decision rule :*

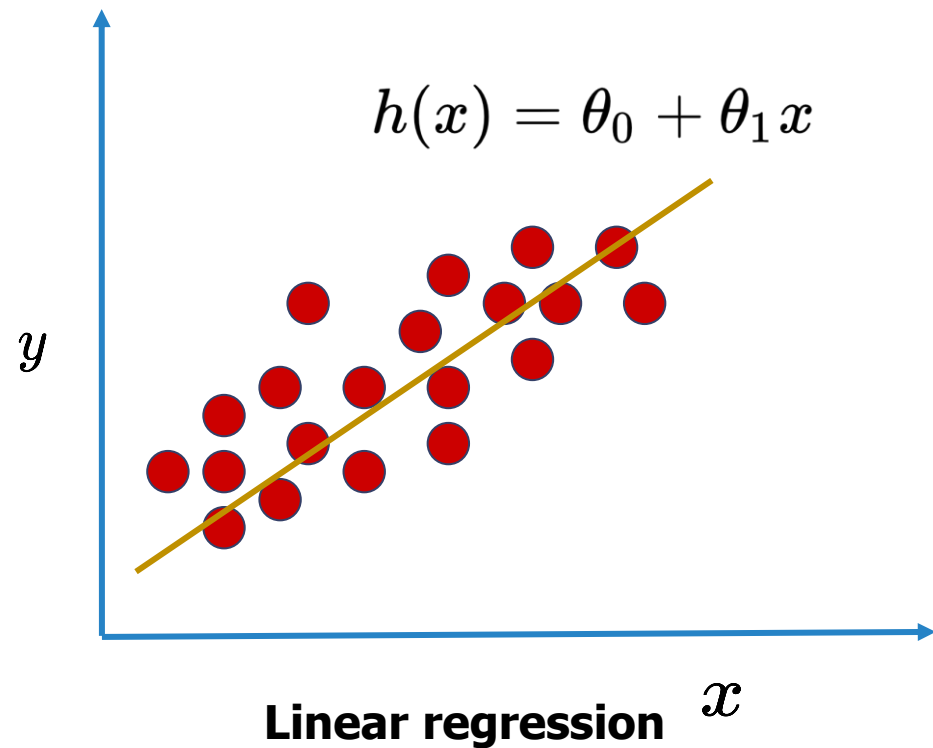
$$y = \begin{cases} 1, & \text{if } h(\vec{x}^{(i)}) \geq 0.5 \\ 0, & \text{if } h(\vec{x}^{(i)}) < 0.5 \end{cases}$$

→ This is a good customer !

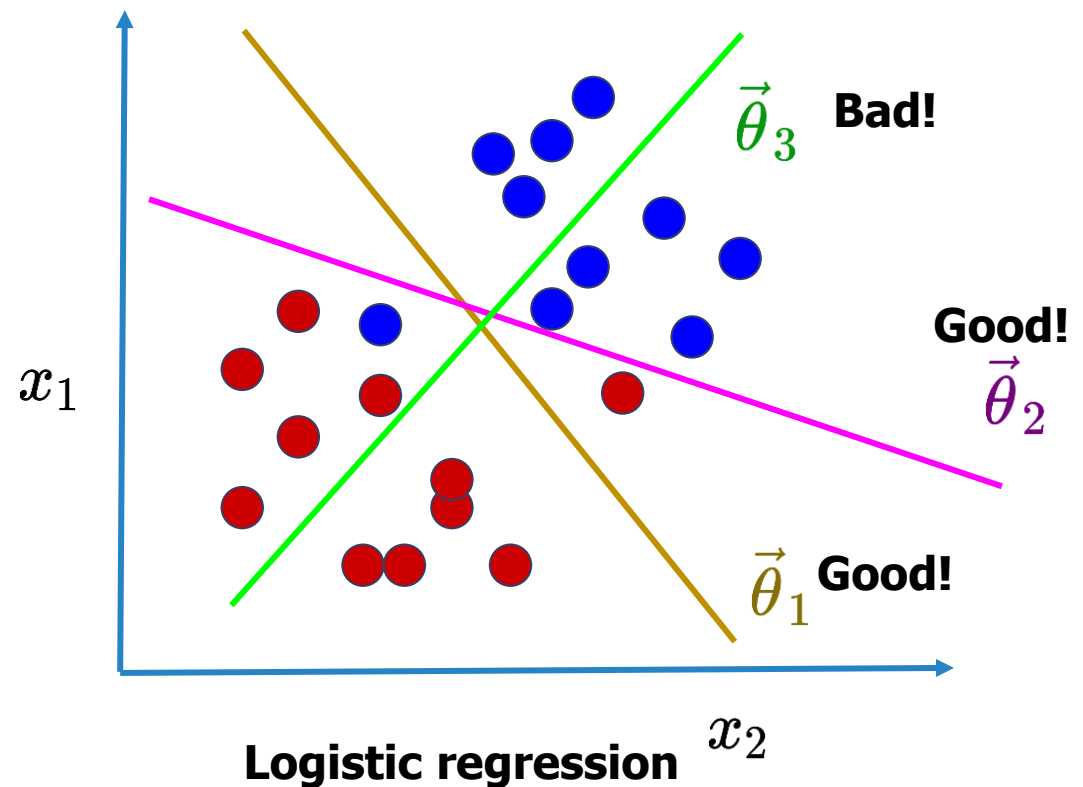
# A simple example



# A simple example



$$h(\vec{x}) = f(z) = f(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$



# Classification cost function

---

# Classification cost function

---

Cost function: sum of errors from classifying sample  $i$ :  $\sum_i E_i$

$$E_i = -(y^{(i)} \log(h(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h(\vec{x}^{(i)})))$$

The cost function is 'cross entropy' error, defining how actual classes match your class predictions.

| $h(\vec{x}^{(i)})$ | $y$ | Cost |
|--------------------|-----|------|
| 0.8                | 1   | 0.10 |
| 0.1                | 1   | 1.00 |
| 0.1                | 0   | 0.05 |

# Classification cost function (IN-DEPTH ANALYSIS)

---

Suppose to have at disposal a dataset  $\mathcal{D} = \{(\boldsymbol{\varphi}(1), y(1)), \dots, (\boldsymbol{\varphi}(N), y(N))\}$  where  $\boldsymbol{\varphi}(i) \in \mathbb{R}^{d \times 1}$  and  $y(i) \in \{0, 1\}$ ,  $i = 1, \dots, N$ , *i.i.d.*

Estimate a logistic regression model  $P(y(i) = 1 | \boldsymbol{\varphi}(i)) = \frac{1}{1 + e^{-\boldsymbol{\varphi}(i)^\top \boldsymbol{\theta}}} \equiv \pi(i)$

The **logistic regression cost function**  $J(\boldsymbol{\theta})$  is defined as:

$$J(\boldsymbol{\theta}) = - \sum_{i=1}^N (y(i) \cdot \ln \pi(i) + (1 - y(i)) \cdot \ln[1 - \pi(i)])$$

# Classification cost function (IN-DEPTH ANALYSIS)

## Cost function interpretation

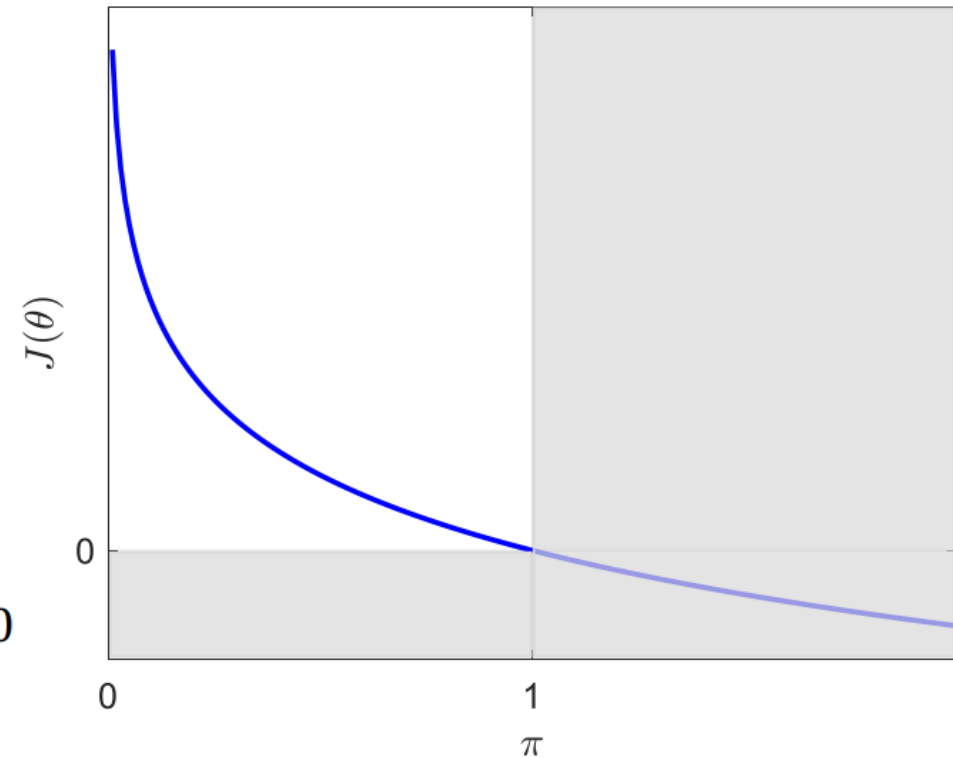
Suppose there is only one datum  $\mathcal{D} = \{(\boldsymbol{\varphi}, y)\}$

$$\Rightarrow J(\boldsymbol{\theta}) = \begin{cases} -\ln \pi & \text{if } y = 1 \\ -\ln[1 - \pi] & \text{if } y = 0 \end{cases}$$

### Case $y = 1$

$J(\boldsymbol{\theta}) = -\ln \pi$

- $J(\boldsymbol{\theta}) \approx 0$  if  $y = 1$  and  $\pi \approx 1$
- $J(\boldsymbol{\theta}) \approx +\infty$  if  $y = 1$  and  $\pi \approx 0$



# Classification cost function (IN-DEPTH ANALYSIS)

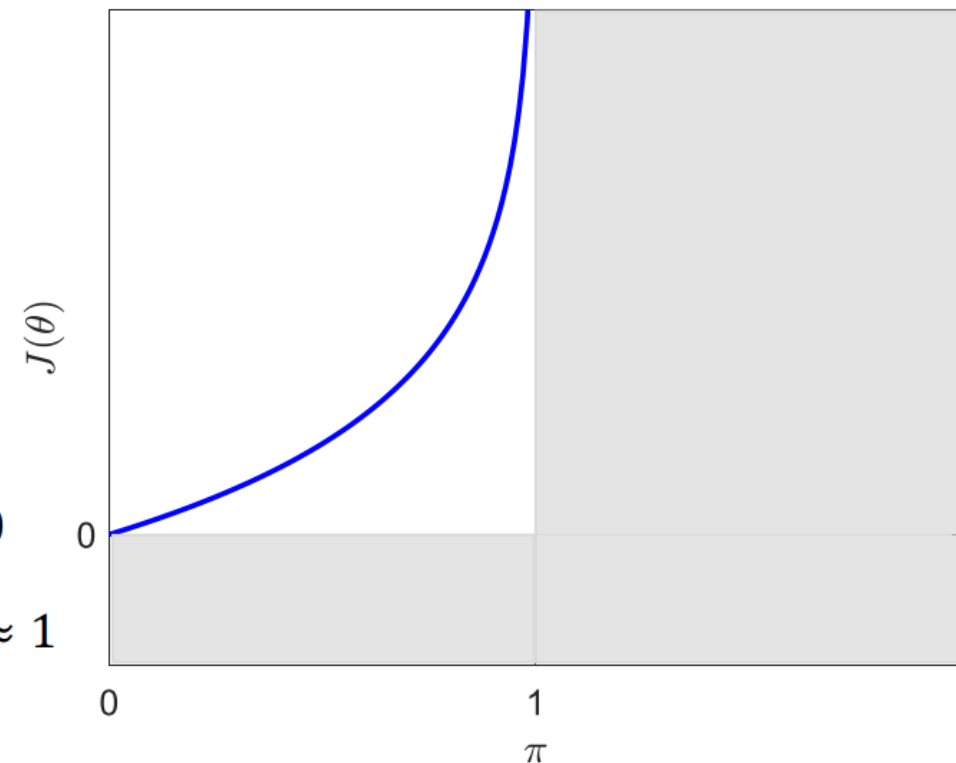
## Cost function interpretation

Suppose there is only one datum  $\mathcal{D} = \{(\boldsymbol{\varphi}, y)\}$

$$\Rightarrow J(\boldsymbol{\theta}) = \begin{cases} -\ln \pi & \text{if } y = 1 \\ -\ln[1 - \pi] & \text{if } y = 0 \end{cases}$$

### Case $y = 0$

- $J(\boldsymbol{\theta}) = -\ln[1 - \pi]$
- $J(\boldsymbol{\theta}) \approx 0$  if  $y = 0$  and  $\pi \approx 0$
  - $J(\boldsymbol{\theta}) \approx +\infty$  if  $y = 0$  and  $\pi \approx 1$



# Classification cost function (IN-DEPTH ANALYSIS)

We have to compute the gradient of  $J(\boldsymbol{\theta})$  with respect to  $\boldsymbol{\theta} \in \mathbb{R}^{d \times 1}$ . First, compute the

derivative of  $s(a) = \frac{1}{1+e^{-a}}$

$$\frac{\partial s(a)}{\partial a} = \frac{\partial}{\partial a} \left[ \frac{1}{1+e^{-a}} \right] = \frac{\partial}{\partial a} [(1+e^{-a})^{-1}] = -(1+e^{-a})^{-2} (e^{-a})(-1) = -(1+e^{-a})^{-2} (-e^{-a}) = \frac{e^{-a}}{(1+e^{-a})^2}$$

$$= \frac{1}{(1+e^{-a})} \cdot \frac{e^{-a}}{(1+e^{-a})} = \frac{1}{(1+e^{-a})} \cdot \frac{(1+e^{-a}) - 1}{1+e^{-a}} = \frac{1}{1+e^{-a}} \cdot \left( \frac{1+e^{-a}}{1+e^{-a}} - \frac{1}{1+e^{-a}} \right) = s(a) \cdot [1 - s(a)]$$

In the case where  $a = \boldsymbol{\varphi}^\top \boldsymbol{\theta}$ , we have that

$$\underbrace{\nabla_{\boldsymbol{\theta}} s(\boldsymbol{\varphi}^\top \boldsymbol{\theta})}_{d \times 1} = \underbrace{\boldsymbol{\varphi}}_{d \times 1} \cdot \underbrace{s(\boldsymbol{\varphi}^\top \boldsymbol{\theta})}_{1 \times 1} \cdot \underbrace{[1 - s(\boldsymbol{\varphi}^\top \boldsymbol{\theta})]}_{1 \times 1} = \boldsymbol{\varphi} \cdot \pi \cdot [1 - \pi]$$

# Classification cost function (IN-DEPTH ANALYSIS)

---

We can now compute the gradient of  $J(\boldsymbol{\theta})$

$$J(\boldsymbol{\theta}) = - \sum_{i=1}^N (y(i) \ln \pi(i) + (1 - y(i)) \ln[1 - \pi(i)]) \quad \pi(i) = \frac{1}{1 + e^{-\boldsymbol{\varphi}(i)^T \boldsymbol{\theta}}}$$

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = - \sum_{i=1}^N \left( y(i) \frac{\pi'(i)}{\pi(i)} + (1 - y(i)) \frac{-\pi'(i)}{1 - \pi(i)} \right) = - \sum_{i=1}^N \left( y(i) \frac{\boldsymbol{\varphi}(i) \pi(i) [1 - \pi(i)]}{\pi(i)} + (1 - y(i)) \frac{-\boldsymbol{\varphi}(i) \pi(i) [1 - \pi(i)]}{1 - \pi(i)} \right)$$

$$= \sum_{i=1}^N ( -y(i) \boldsymbol{\varphi}(i) [1 - \pi(i)] - (1 - y(i)) (-\boldsymbol{\varphi}(i) \pi(i)) ) = \sum_{i=1}^N ( \boldsymbol{\varphi}(i) \cdot [-y(i) + y(i) \pi(i)] + \boldsymbol{\varphi}(i) \cdot [\pi(i) - y(i) \pi(i)] )$$

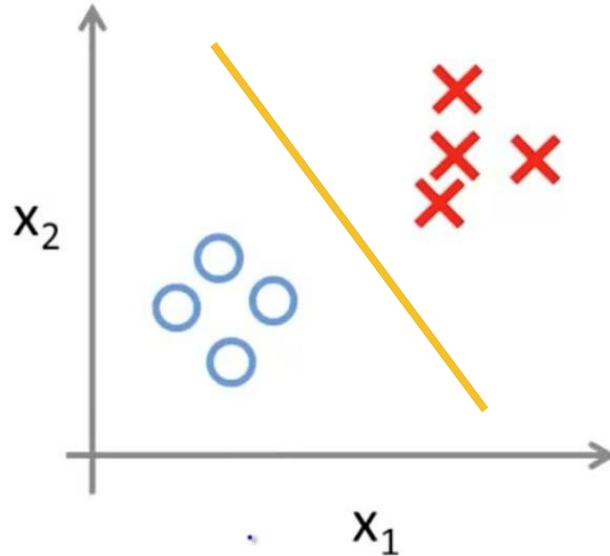
$$= \sum_{i=1}^N ( \boldsymbol{\varphi}(i) \cdot [-y(i) + y(i) \pi(i) - y(i) \pi(i) + \pi(i)] ) = \sum_{i=1}^N \underset{d \times 1}{\boldsymbol{\varphi}(i)} \cdot \underset{1 \times 1}{(\pi(i) - y(i))}$$

# Classification

**Classification** is the problem of assigning observations to one or more categories.

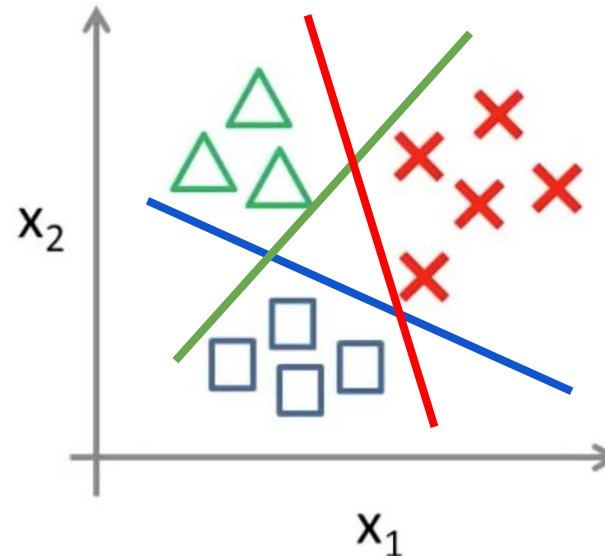
## Binary Classification

Involves only 2 classes



## Multiclass Classification

Involves more than 2 classes



### Examples

#### Binary

- Spam detection
- Churn/no churn customer retention
- Develop Diabetes/don't
- Default/repay loan

#### Multiclass

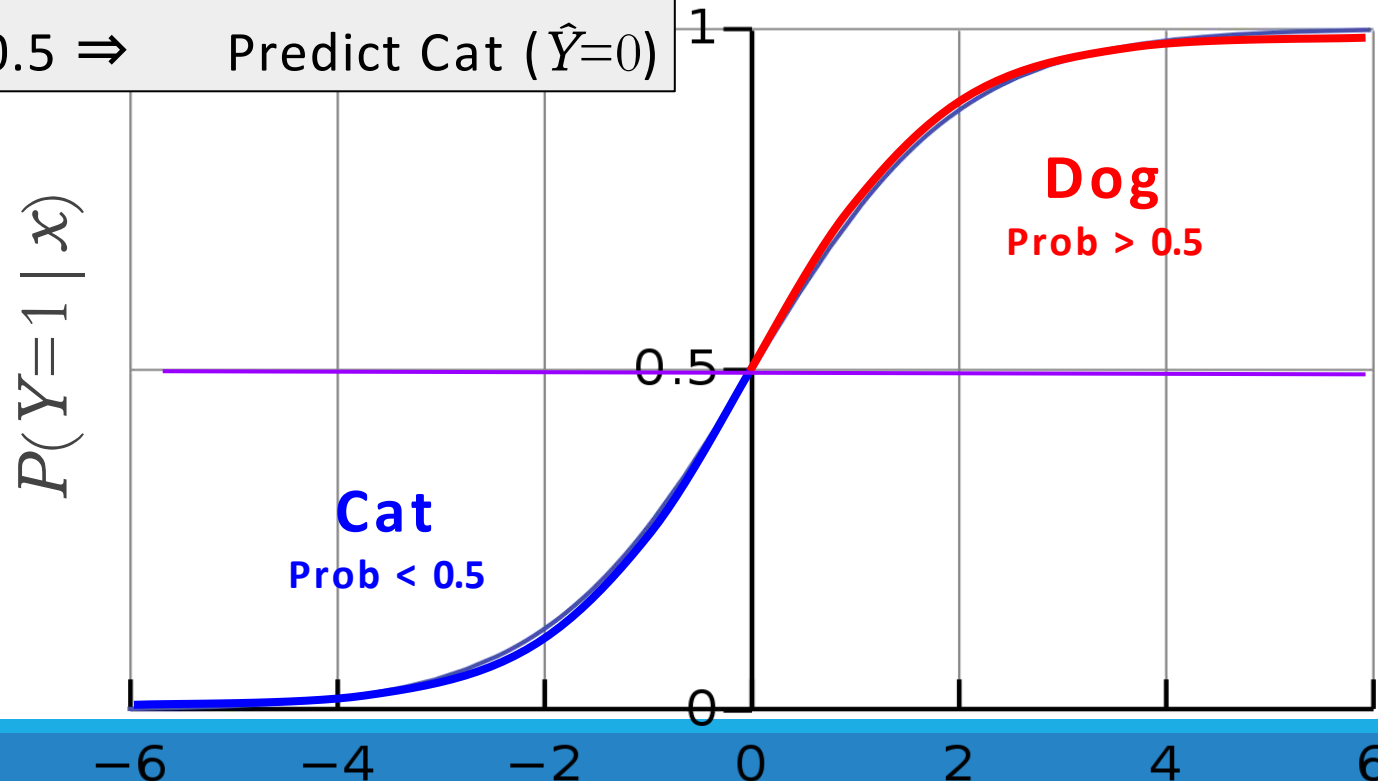
- Image recognition
- Natural language processing

# Logistic Regression Threshold

We can select some threshold (**Prob = 0.5**)

- If  $P(Y=1 | x) > 0.5 \Rightarrow$  Predict Dog ( $\hat{Y}=1$ )
- If  $P(Y=1 | x) < 0.5 \Rightarrow$  Predict Cat ( $\hat{Y}=0$ )

$$P(Y = 1|x) = \frac{1}{1+e^{-(w_0+w_1x)}}$$



# Multiclass Classification

---

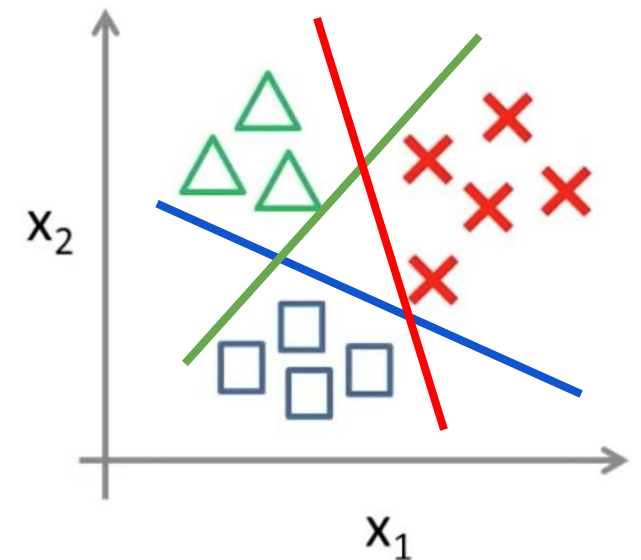
Logistic regression can be applied to solve multiclass problems.

## Common Approaches

1. One-vs-Rest (One-vs-All)
2. Softmax Regression (Multinomial Logistic Regression)

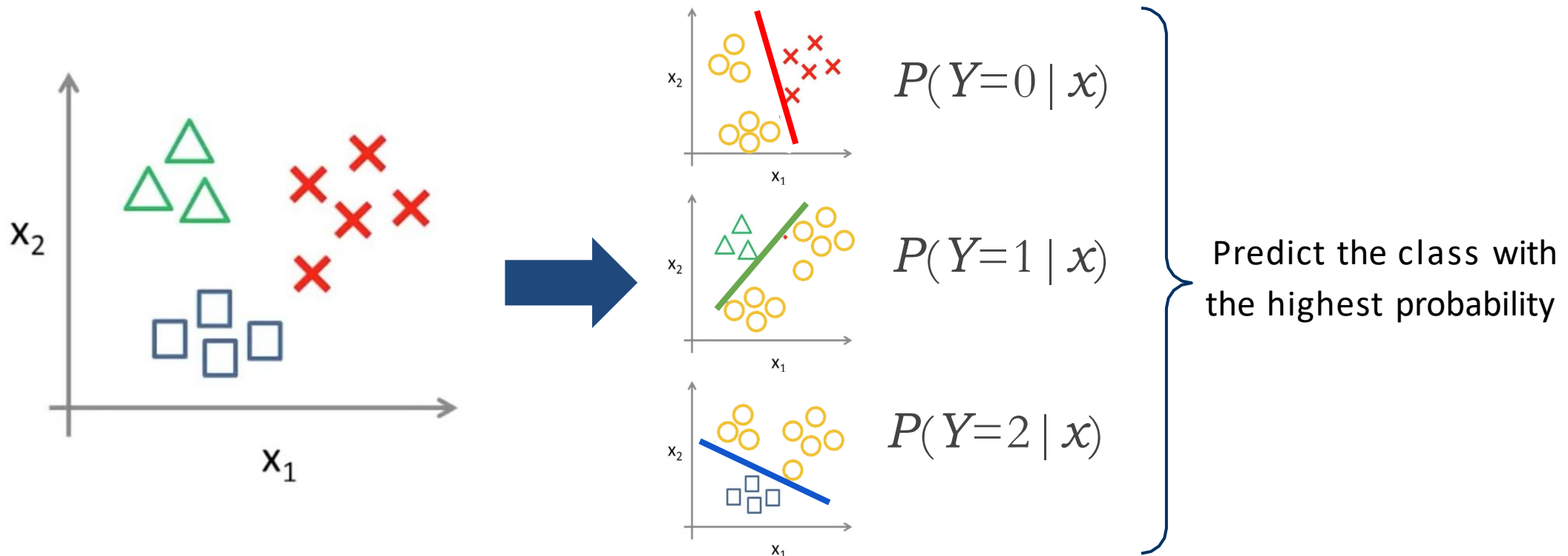
## Multiclass Classification

Involves more than 2 classes



# One-vs-Rest (One-vs-All)

For each class, build a logistic regression to find the probability the observation belongs to that class. For each data point, predict the class with the highest probability.



# Softmax Regression (Multinomial Logistic Regression)

In softmax regression the probability that a data point belongs to each class is calculated by:

$$\begin{bmatrix} P(Y = 1|x; \theta) \\ P(Y = 2|x; \theta) \\ \vdots \\ P(Y = K|x; \theta) \end{bmatrix} = \underbrace{\frac{1}{\sum_{j=1}^K \exp(\theta^{(j)\top} x)}}_{\text{Normalizes probabilities so they sum to 1.}} \begin{bmatrix} \exp(\theta^{(1)\top} x) \\ \exp(\theta^{(2)\top} x) \\ \vdots \\ \exp(\theta^{(K)\top} x) \end{bmatrix} \rightarrow \text{Predict the class with the highest probability}$$

Separate  $\theta^{(j)} \in \mathbb{R}^d$  for each class

If  $K=2$ , softmax regression reduces to the same binary logistic regression formulas we saw earlier. Check out this [overview of softmax regression](#) for the proof.

The standard (unit) softmax function  $\sigma : \mathbb{R}^K \mapsto (0, 1)^K$ , where  $K \geq 1$ , is defined by the formula

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } i = 1, \dots, K \text{ and } \mathbf{z} = (z_1, \dots, z_K) \in \mathbb{R}^K.$$

# Linearity check for logistic

---

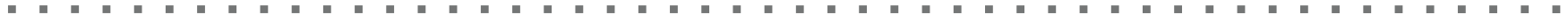
There are key assumptions on data that can be used to check if linear regression is appropriate for the data.

- Linear Relationship between the features and target
- Little or no Multicollinearity between the features
- No High Leverage Points or Influential Outliers
- No auto-correlation
  - "There is no temporal correlation, meaning that the values occurring at one time may have no relationship with the values occurring at a previous time."

# Classification Performance

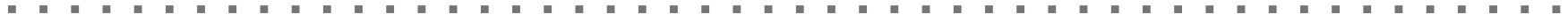
---

# A Quote



**We have achieved 99% accuracy on our COVID-19 patient screening.**

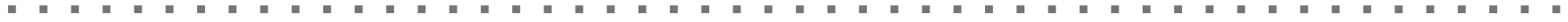
# A Quote



**We have achieved 99% accuracy on our COVID-19 patient screening app.**

What is wrong with this quote?

# A Quote



**We have achieved 99% accuracy on our COVID-19 patient screening app.**

What is wrong with this quote?

- Sample size
- Input Distribution
- Result Distribution

# Metrics



Different problems require different metrics

- Regression problem
- Classification problem
- Object Detection

# Metrics



Different problems require different metrics

- Regression problem
  - Sum of squared error (SSE)
  - Mean squared error (MSE)
  - Root mean squared error (RMSE)
- Classification problem
- Object Detection

# Metrics



Different problems require different metrics

- Regression problem
- Classification problem
  - Confusion Matrix
  - Accuracy, Precision, Recall, F1-Score
  - ROC
- Object Detection

# Metrics



Different problems require different metrics

- Regression problem
- Classification problem
- Object Detection (we will cover this topic in Deep Learning course)
  - Intersect over Union (IOU)
  - Mean Average Precision (mAP)

# Classification Metrics



## Confusion Matrix

A table that explains the result of predictions compared to the actual result.

Predict = What the algorithm predicts

Actual = The actual value

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

# Classification Metrics - Confusion Matrix



## Confusion Matrix

A table that explains the result of predictions compared to the actual result.

Positive Actual Value = number of positive instances

Negative Actual Value = number of negative instance

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

# Classification Metrics - Confusion Matrix



## Confusion Metrics

A table that explains the result of predictions compared to the actual result.

Positive Prediction = number of instances the algorithm predicts as positive

Negative Prediction = number of instances the algorithm predicts as negative

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

# Classification Metrics - Confusion Matrix



True Positive

- Positive value that is predicted as positive

True Negative

- Negative value that is predicted as negative

False Positive (Type I error)

- Negative value that is predicted as positive

False Negative (Type II error)

- Positive value that is predicted as negative

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

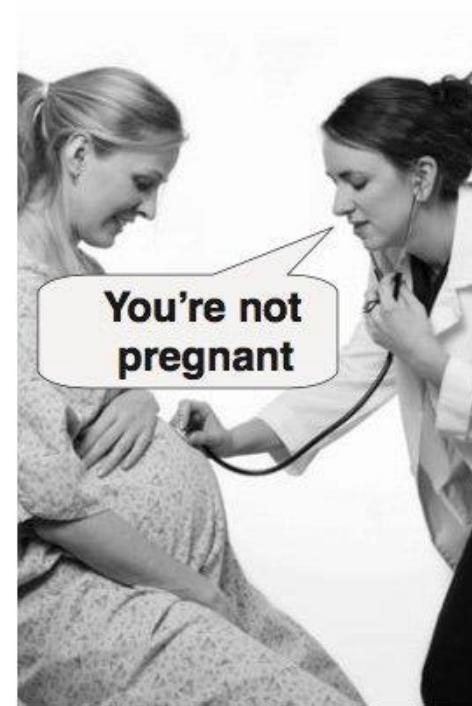
# Classification Metrics - Confusion Matrix



**Type I error**  
(false positive)



**Type II error**  
(false negative)



# Classification Metrics – Acc, Pre, Rec



$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

What percentage can the algorithm predict correctly?

$$Precision = \frac{TP}{TP + FP}$$

Out of all predicted positives, what percentage can the algorithm predict correctly?

$$Recall = \frac{TP}{TP + FN}$$

Out of all actual positives, what percentage can the algorithm predict correctly?

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

# Classification: Binary classification

.....

## Example

|                  |     | Actual values |     |
|------------------|-----|---------------|-----|
|                  |     | Cat           | Dog |
| Predicted values | Cat | 5             | 2   |
|                  | Dog | 3             | 3   |

$$\text{Accuracy} = \frac{(5 + 3)}{13} = \frac{8}{13} = 0.62$$

$$\text{Misclassification Rate} = \frac{(2 + 3)}{13} = \frac{5}{13} = 0.38$$

$$\text{Recall} = \frac{5}{5 + 3} = \frac{5}{8} = 0.625$$

$$\text{Precision} = \frac{5}{5 + 2} = \frac{5}{7} = 0.714$$

# Classification: Multiclass classification

## Example

|                  |            | Actual values |            |           |
|------------------|------------|---------------|------------|-----------|
|                  |            | setosa        | versicolor | virginica |
| Predicted values | setosa     | 10            | 2          | 4         |
|                  | versicolor | 1             | 16         | 1         |
|                  | virginica  | 0             | 2          | 9         |

$$\text{Recall}_{\text{virginica}} = ?$$

$$\text{Precision}_{\text{virginica}} = ?$$

$$\text{Accuracy} = \frac{(10 + 16 + 9)}{45} = \frac{35}{45} = 0.78$$

$$\text{Misclassification Rate} = 1 - 0.78 = 0.22$$

$$\text{Recall}_{\text{setosa}} = \frac{10}{10 + 1 + 0} = \frac{10}{11} = 0.91$$

$$\text{Precision}_{\text{setosa}} = \frac{10}{10 + 2 + 4} = \frac{10}{16} = 0.625$$

$$\text{Recall}_{\text{versicolor}} = \frac{16}{2 + 16 + 2} = \frac{16}{20} = 0.8$$

$$\text{Precision}_{\text{versicolor}} = \frac{16}{1 + 16 + 1} = \frac{16}{18} = 0.89$$

# Classification Metrics



Let's try to understand this:

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

Company S has developed a test kit for screening virus C. Upon testing, its confusion matrix looks like this.

|           |          | Actual   |          |
|-----------|----------|----------|----------|
|           |          | Positive | Negative |
| Predicted | Positive | 60       | 50       |
|           | Negative | 40       | 850      |

- a) How many tests are there?
- b) How many predicted positives?
- c) How many predicted negatives?
- d) Accuracy?
- e) Precision?
- f) Recall?
- g) Should we use it?

# Classification Metrics



Let's try to understand this:

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

Company S has developed a test kit for screening virus C. Upon testing, its confusion matrix looks like this.

|           |          | Actual   |          |
|-----------|----------|----------|----------|
|           |          | Positive | Negative |
| Predicted | Positive | 60       | 50       |
|           | Negative | 40       | 850      |

In medical field, scientists need more tools to analyze.

$$\text{Sensitivity} = \text{True Positive Rate} = \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Specificity} = \text{True Negative Rate} = \frac{TN}{TN + FP}$$

# Classification Metrics – F1 Score



$$F_1 = \frac{2 * precision * recall}{precision + recall}$$

$F_1$  score measures a test's accuracy using the combination of precision and recall.

It ranges between 0 and 1.

- 0 if either precision or recall is zero
- 1 if the prediction has perfect precision and recall

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

# Classification Metrics – ROC Curve



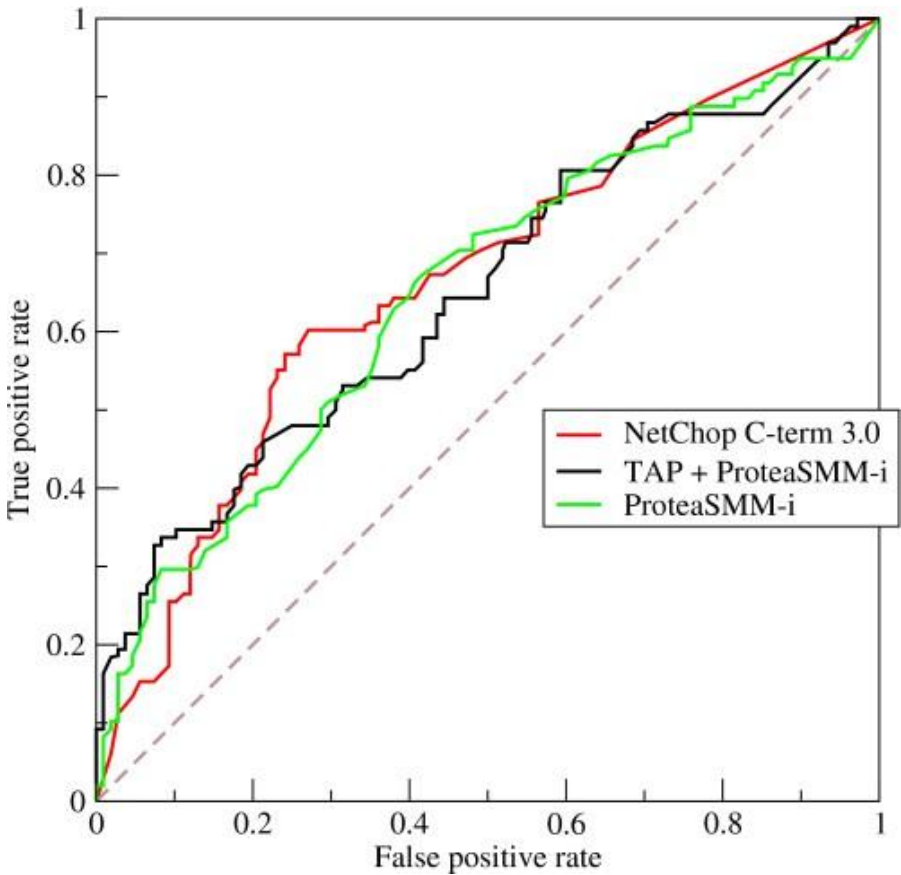
ROC Curve (Receiver Operating Characteristics Curve)

- A curve that shows relationships between TPR and FPR when discrimination threshold is varied.

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |



[https://en.wikipedia.org/wiki/Receiver\\_operating\\_characteristic#/media/File:Roccurves.png](https://en.wikipedia.org/wiki/Receiver_operating_characteristic#/media/File:Roccurves.png)

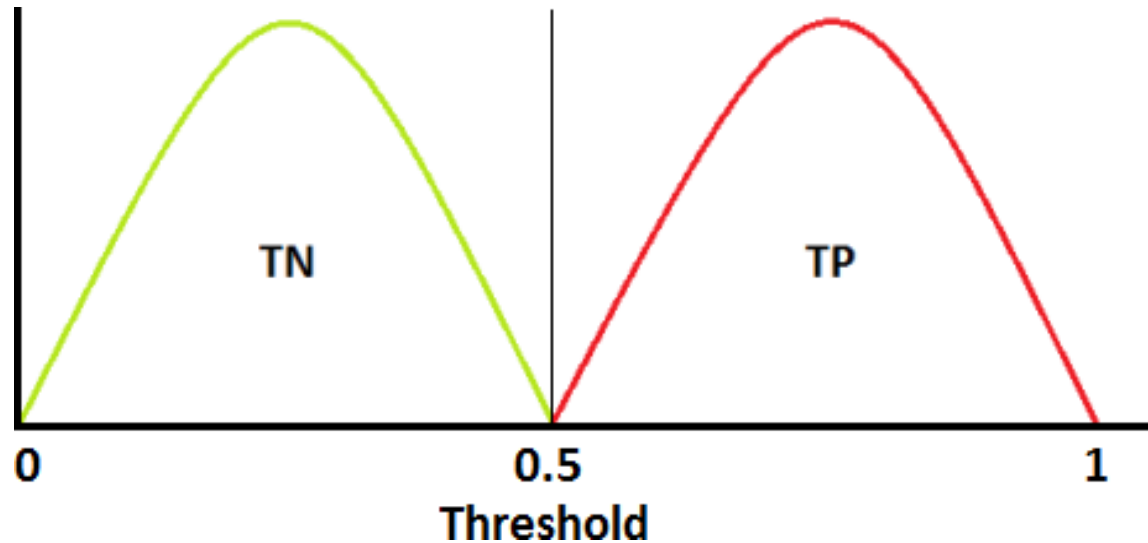
[https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc\\_curve.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html)

[https://scikit-learn.org/1.0/auto\\_examples/model\\_selection/plot\\_roc.html](https://scikit-learn.org/1.0/auto_examples/model_selection/plot_roc.html)

# Classification Metrics – ROC Curve



If we select threshold = 0.5, the result will look like this.



|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

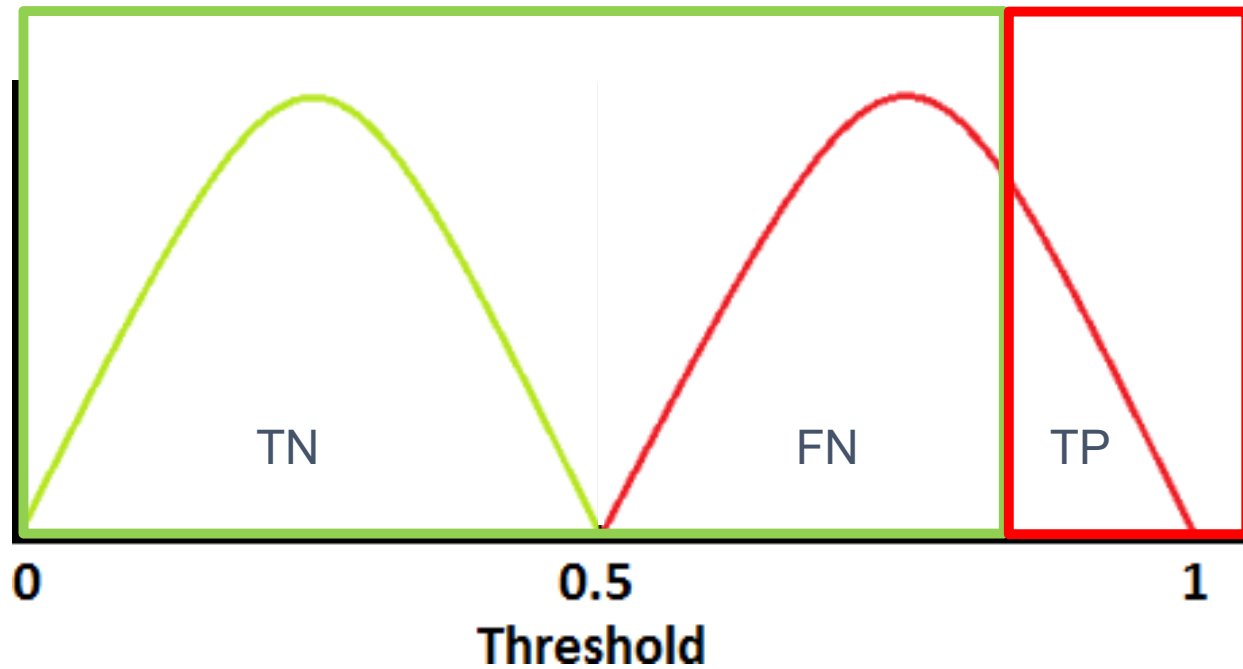
$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

# Classification Metrics – ROC Curve



If we select threshold = 0.8, the result will look like this.



|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

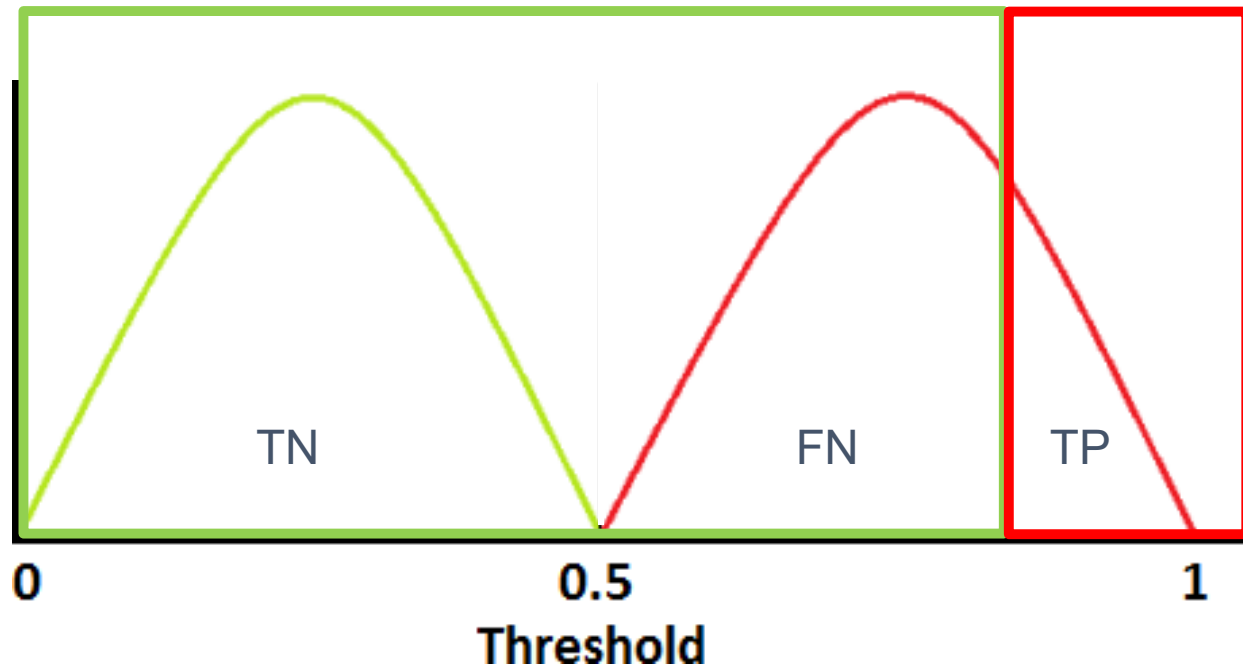
$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

# Classification Metrics – ROC Curve



If we select threshold = 0.8, the result will look like this.



$$TPR < 1$$

$$FPR = 0$$

|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

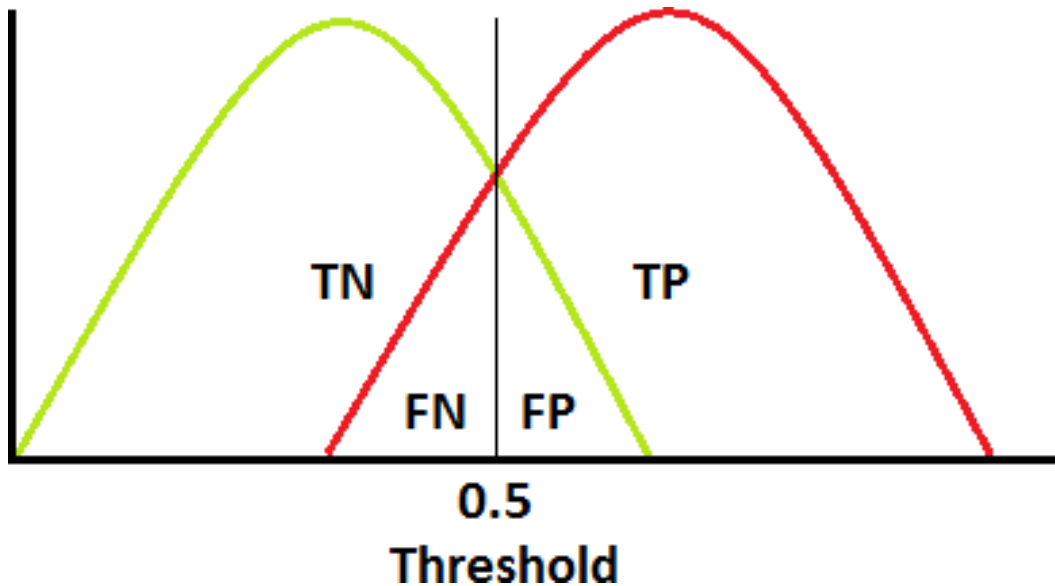
$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

# Classification Metrics – ROC Curve



What if the data distributions overlap?



|           |          | Actual         |                |
|-----------|----------|----------------|----------------|
|           |          | Positive       | Negative       |
| Predicted | Positive | True Positive  | False Positive |
|           | Negative | False Negative | True Negative  |

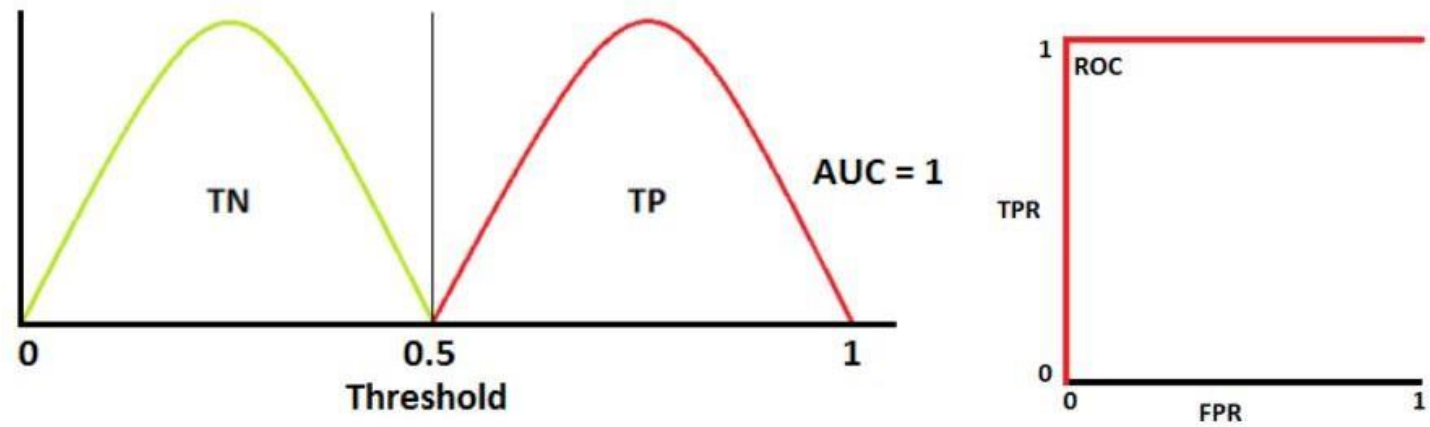
$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

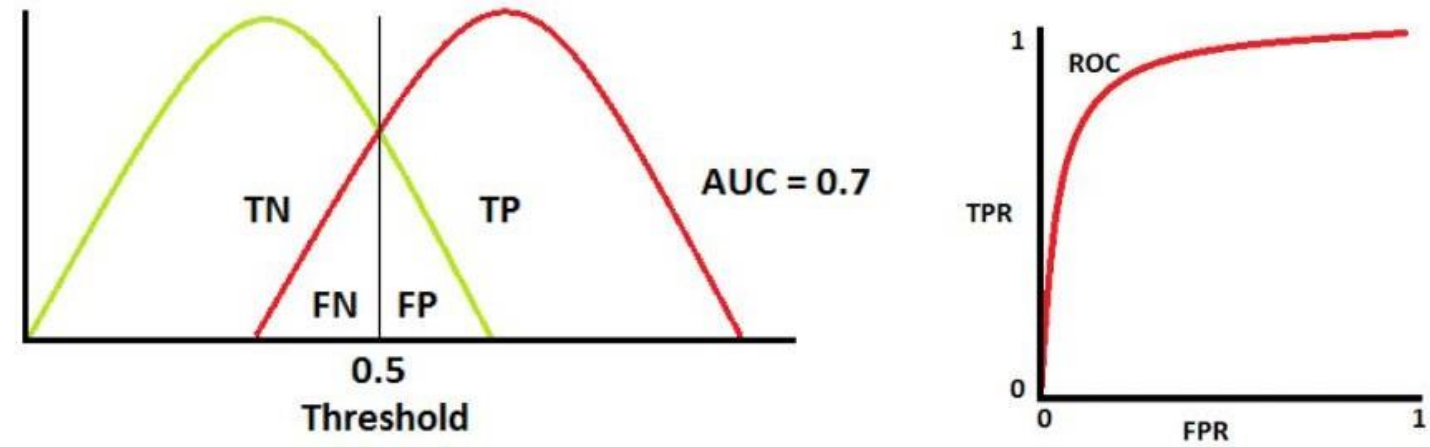
# Classification Metrics – ROC Curve



Perfect ROC



ROC that we will mostly get

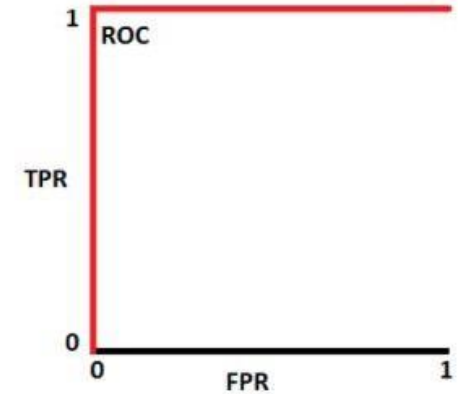
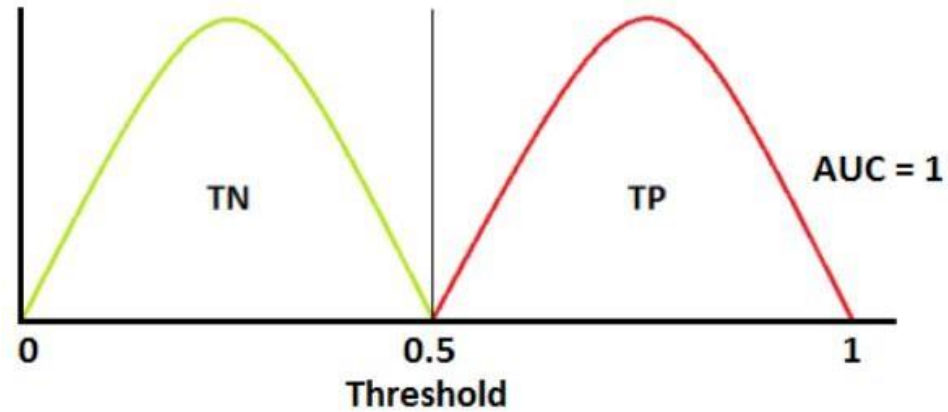


# Classification Metrics – ROC Curve



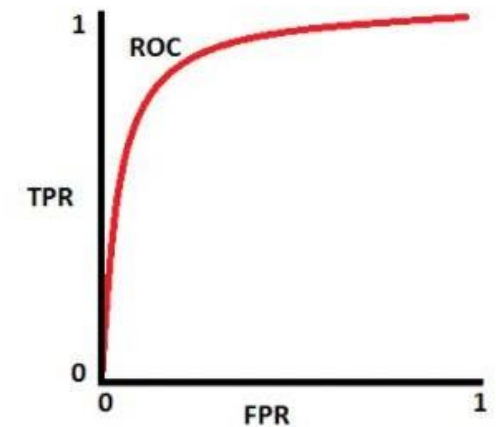
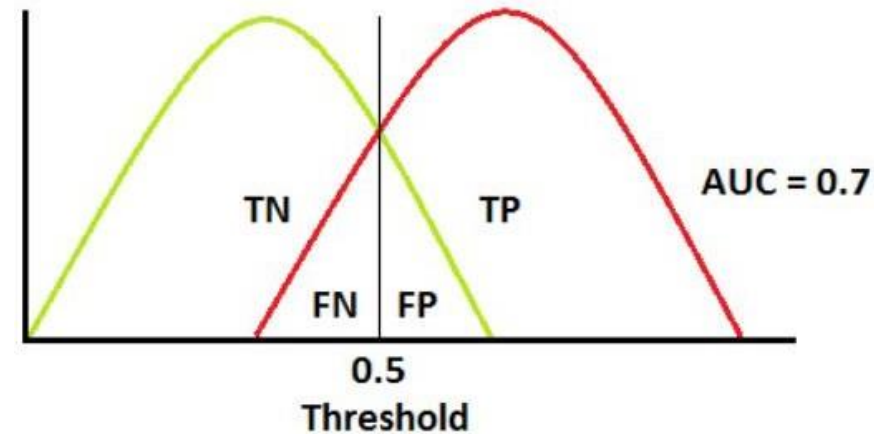
Perfect ROC

- Area Under Curve (AUC) is 1



ROC that we will mostly get

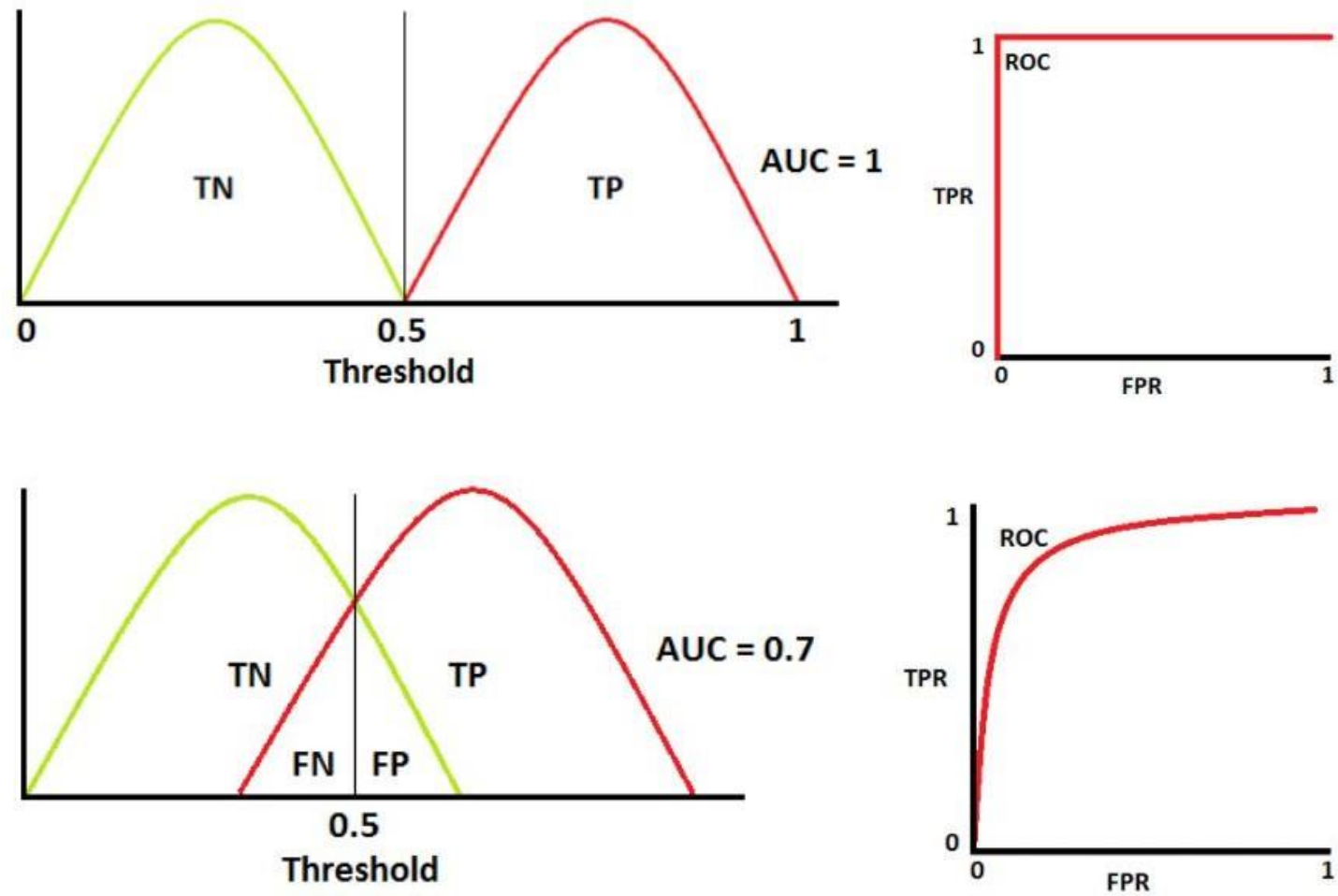
- Area Under Curve (AUC) is  $< 1$



# Classification Metrics – AUC



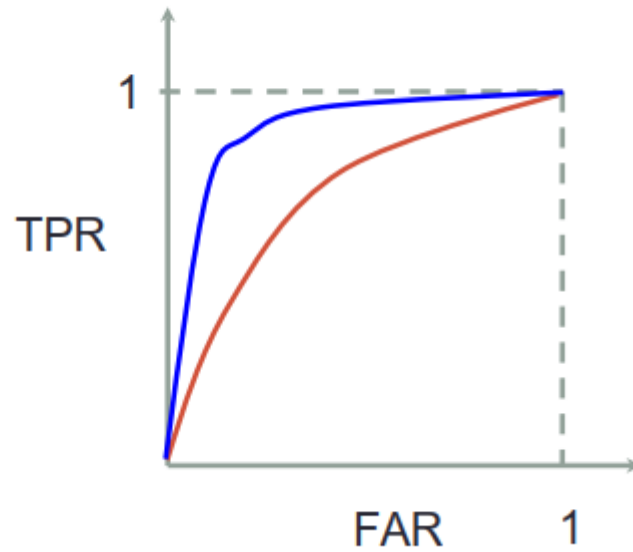
Given 2 models, model with higher AUC performs better.



# Classification Metrics – AUC

.....

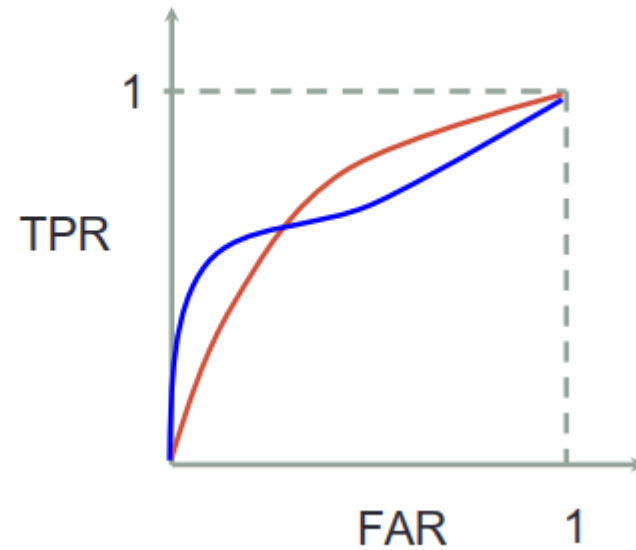
- Which is better?



# Classification Metrics – AUC

.....

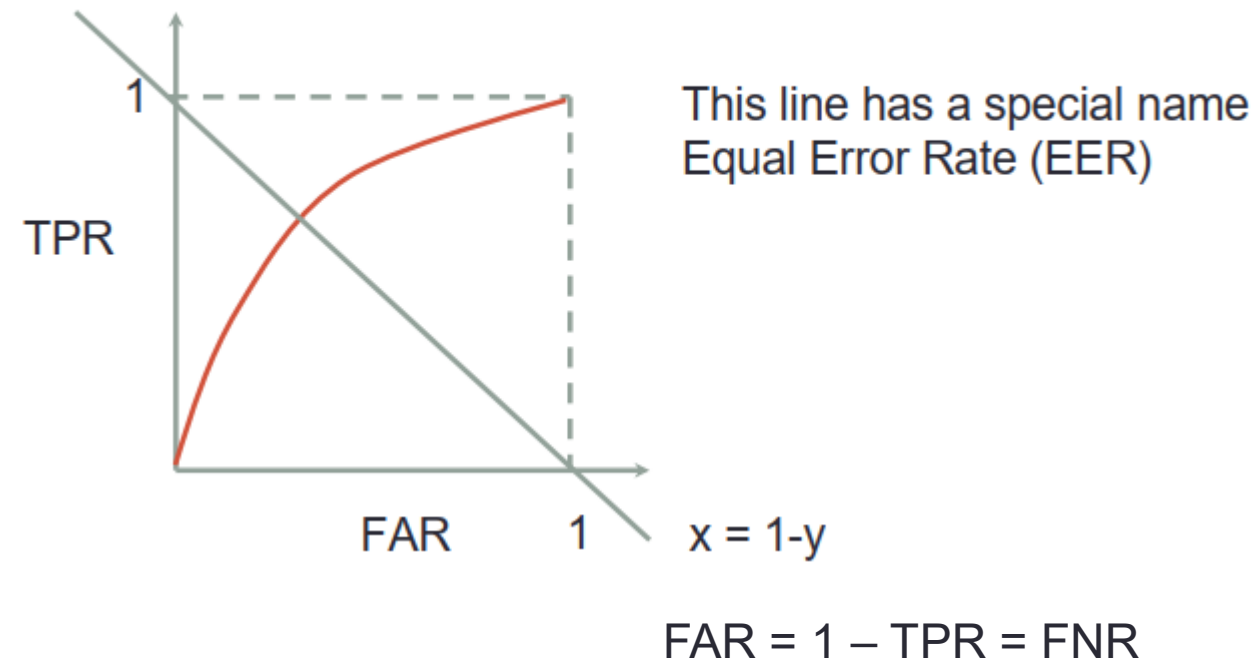
- Which is better?



# Selecting the threshold

---

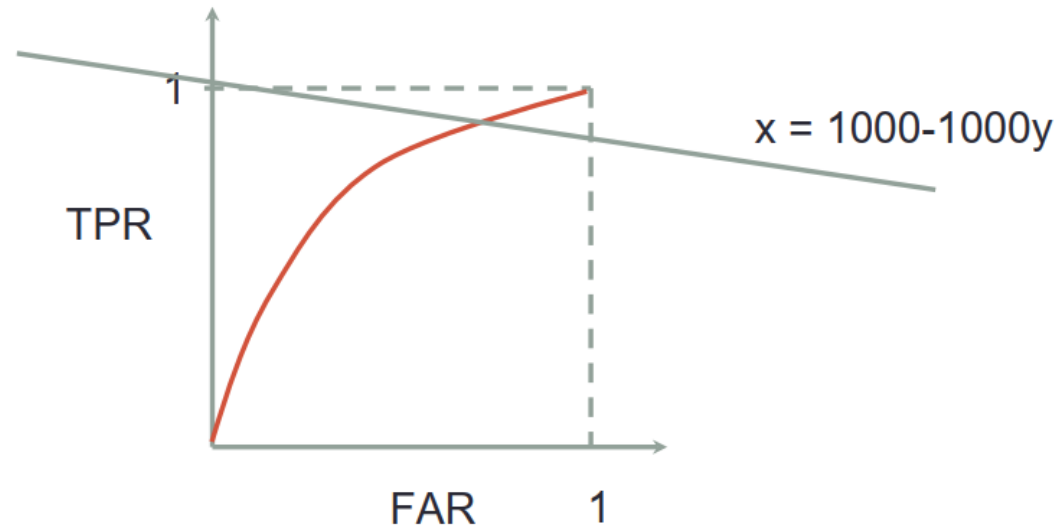
- Select based on the application
- Trade off between TP and FA. Know your application, know your users



# Selecting the threshold

.....

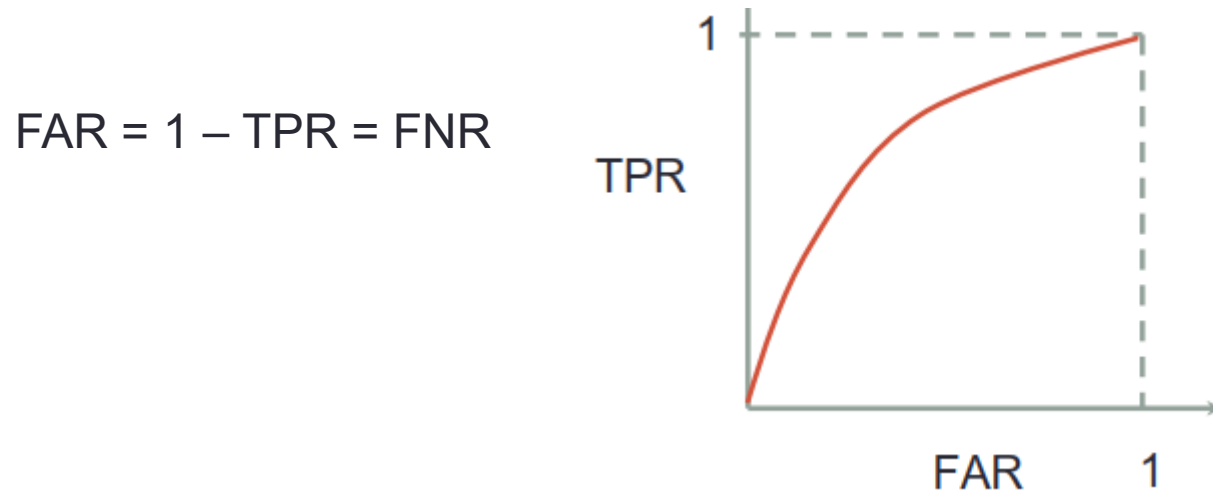
- Select based on the application
- Trade off between TP and FA. Know your application, know your users.
- Is the application about safety?
  - A miss is 1000 times more costly than false alarm.  
 $\text{FAR} = 1000(1 - \text{TPR}) \Rightarrow x = 1000 - 1000y$



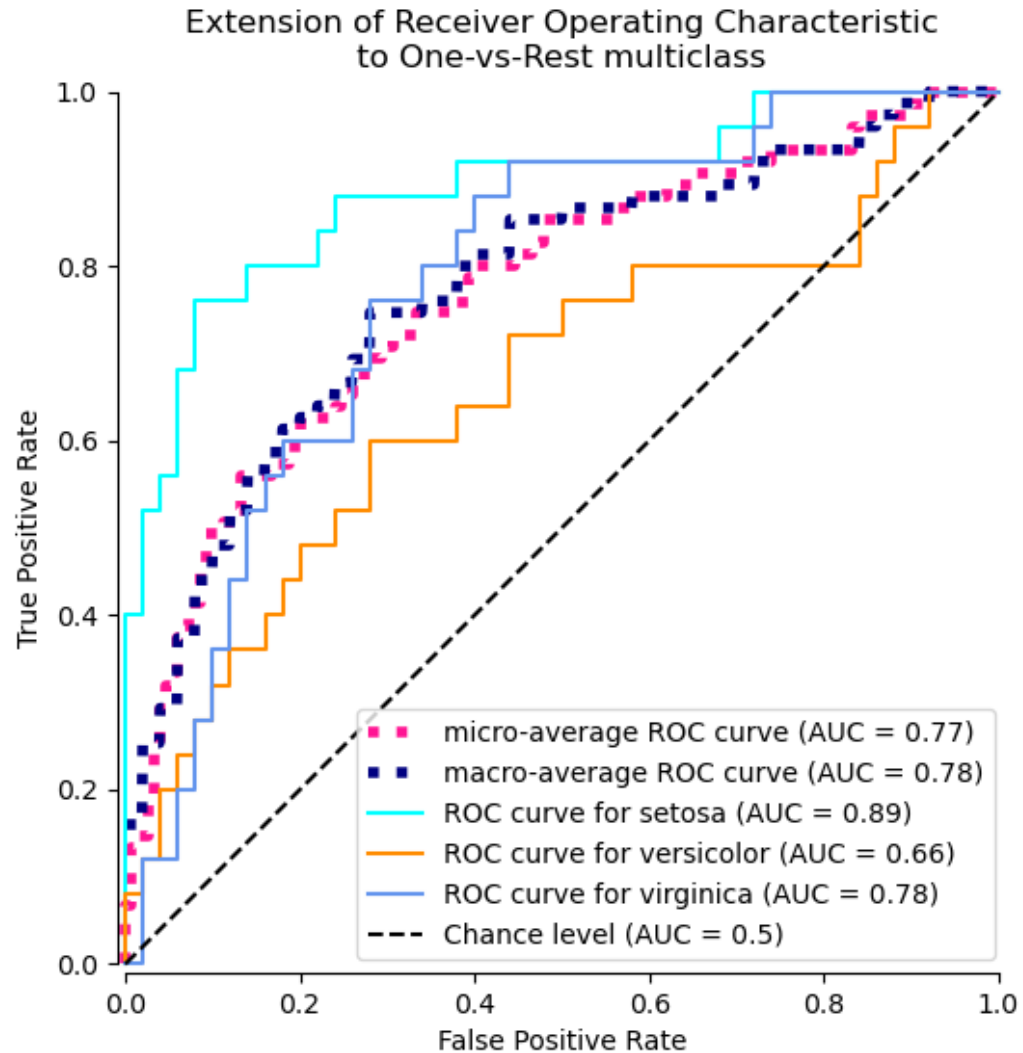
# Selecting the threshold

- Select based on the application
- Trade off between TP and FA. Know your application, know your users.
- Churn prediction
- Predict whether a customer will stop subscription, so we can send a promotional ad.
  - Usual subscription fee 50
  - Cost of calling the customer 5
  - Promotional subscription fee 25

Describe the strategy to pick the threshold

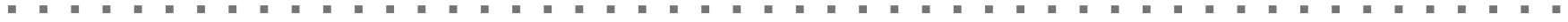


# ROC for Multiclass



[https://scikit-learn.org/stable/auto\\_examples/model\\_selection/plot\\_roc.html](https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html)

# Back to the Quote



**We have achieved 99% accuracy on our COVID-19 patient screening app.**

What is wrong with this quote?

As an ML engineer, how should you train a model to NOT output the quote above?

# Materials

---

## Demo

- [https://colab.research.google.com/drive/1t8iVYn2t0XvSDv1uQ6f6vyDn1bIYkvHr?usp=drive\\_link](https://colab.research.google.com/drive/1t8iVYn2t0XvSDv1uQ6f6vyDn1bIYkvHr?usp=drive_link)
- [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)